

1. Which Phase I classes or interfaces were reused unchanged?

GuessingGame, GuessRules, and GuessFeedback carried over to Wordle with their contracts untouched — I didn't have to change a single method signature on any of them to make Wordle fit. GameConfiguration also didn't need to change at all. MastermindGame and MastermindRules kept working against this shared code the whole time, and I reran the original Mastermind tests to make sure nothing drifted.

2. Which Phase I classes or interfaces were modified, and why?

Driver now lets the player choose Mastermind or Wordle, and for Wordle it loads the external word list and fails with a clear message if that file isn't found. GuessRecord picked up a small addition: its constructor now also takes the code length, so it can answer "did this guess win?" on its own instead of relying on the game loop to tell it. That was the one change needed to make it implement GuessFeedback cleanly.

Most of the actual modification happened earlier, during the refactor that pulled the repeated console behavior out of MastermindGame and into GuessingGame. MastermindRules was updated to implement GuessRules, and GuessRecord to implement GuessFeedback. That's what let Wordle plug into the same game loop without copying any of it.

3. Which new types are specific to Wordle?

WordleConfiguration holds the word length and number of attempts. WordListLoader reads the external dictionary file. WordleRules checks whether a guess is valid and scores it letter by letter. WordleFeedback stores the per-letter result. WordleGame handles Wordle's own prompts and picks the secret word from the dictionary.

4. Which Phase I abstractions turned out to be too specific to Mastermind?

GameConfiguration doesn't transfer to Wordle at all — it's built around peg colors and a fixed code length, and Wordle needs a dictionary instead. GuessRecord has the same problem: it only knows how to hold black/white peg counts, and Wordle needs a result per letter, not per peg. Rather than force these into a shared shape, I left the Mastermind classes as they were and wrote separate Wordle classes to hold Wordle's own data.

5. What refactoring improved reuse?

The big one was moving the turn loop into GuessingGame. It owns reading input, rejecting invalid guesses, tracking remaining attempts, HISTORY, detecting the end of a game, and replay. All it needs from the specific game is a GuessRules to validate and score a guess, and a GuessFeedback to display the result and say whether it was a win. Because that loop already existed in a generic enough shape, WordleGame could plug straight into it instead of needing its own copy.

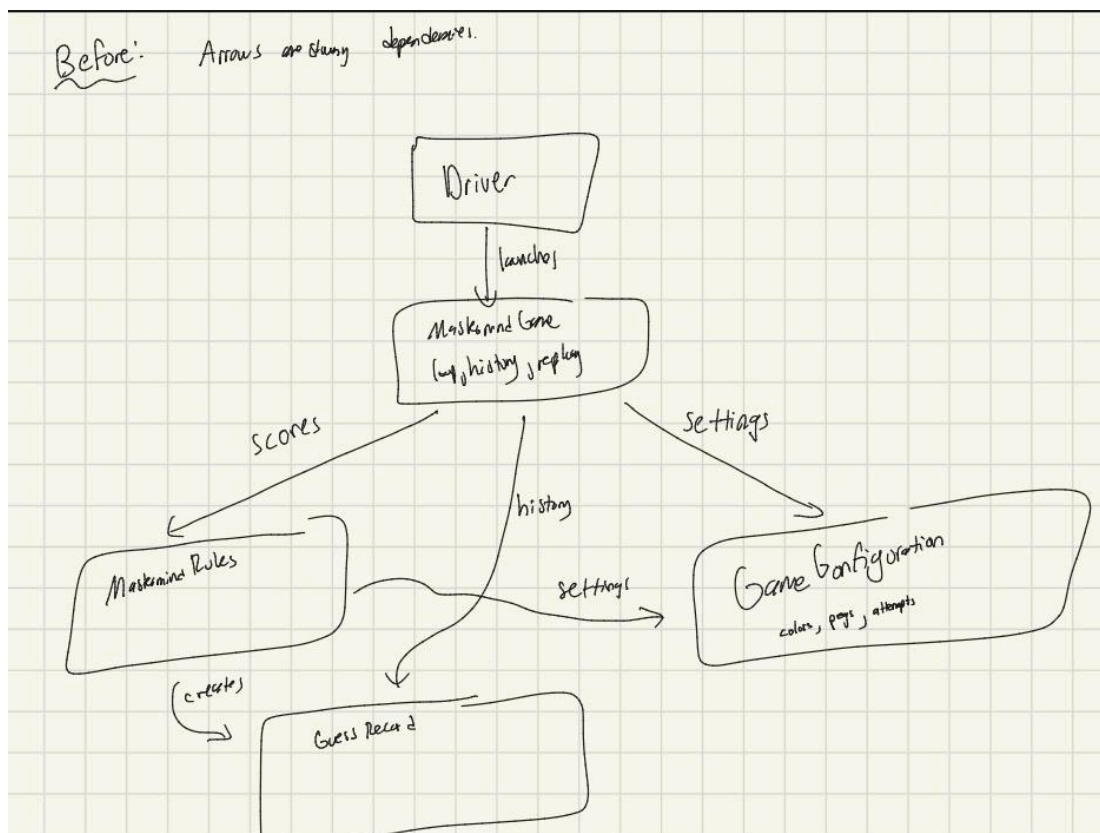
6. Did supporting both games add complexity? Was it worthwhile?

A little — the shared interfaces and the abstract game class are a couple of extra types that Phase I didn't need. I think it paid for itself: the behavior both games have in common now has exactly one implementation, and anything that's genuinely different between them stays in that game's own classes. I specifically avoided building a shared configuration superclass, since Mastermind's settings and Wordle's settings don't actually overlap enough to justify one.

7. What would I design differently if I'd known about Wordle before Phase I?

I'd have named GameConfiguration and GuessRecord more honestly from the start — something like MastermindConfiguration and MastermindFeedback — instead of giving generic names to classes that were really Mastermind-specific. And I'd have designed the shared rules interface, feedback interface, and console loop up front, rather than writing Mastermind as one big class and pulling the reusable parts out of it later.

8. Before-and-after diagrams



8: 2 games, 2 runner

